CAWJ Assignments

Lab1

1. Write a Simple "Hello World" program in java. Understand the build process and also the runtime settings needed to run java programs.
2. Write a program that prints 3 arguments taken from command line.
3. Write a program that prints the sum of 3 numeric arguments taken from command line.
4. Allow the user to enter a string and display the string in alphabetic order and find the length of string.
5. Check whether entered string is a palindrome or not.
6. Write a program in java to count the number of vowels in the input string

Lab2

7. Write a Java program to find the duplicate values of an array of integer values
8. Write a Java program to find the kth smallest and largest element in each array. Elements in the array can be in any order.
9. Write a Java program to move every positive number to the right and every negative number to the left of a given array of integers.
10. Write a Java program to post a question and get a correct answer from the user. Give three chances to answer. If it is correct answer, then display "Good" else display correct answer after 3 attempts.
11. Shown below is a Floyd's triangle.

    1
    2 3
    4 5 6
    7 8 9 10
    11 15
    .
    .
    79 91

    a. Write a program to print this triangle
    b. Modify the program to produce the following form of Floyd's triangle

       1
       0 1
       1 0 1
       0 1 0 1
       1 0 1 0 1

Lab3

12. Design and implement Employee Information System (EIS), comprising of a class called Employee which includes details such as Employee ID, Name, Designation, Salary. The class also provides functionalities such as displaying employee details, searching for an employee based on employee ID, provision for salary increment (10%), etc. The main class of EIS initially collects all employee details and later it provides menu items such as search, salary increment, and display.

13. A book shop maintains the inventory of books that are being sold at the shop. The list includes details such as author, title, price, publisher and stock position. Whenever a customer wants a book, the salesperson inputs the title and author, and the system searches the list and displays whether it is available or not. If it is not, an appropriate message is displayed. If it is, then the system displays the book details and requests for the number of copies required. If the requested copies are available, the total cost of the requested copies is displayed; otherwise, the message "required copies not in stock" is displayed. Design a system using a class called books with suitable member functions and constructors

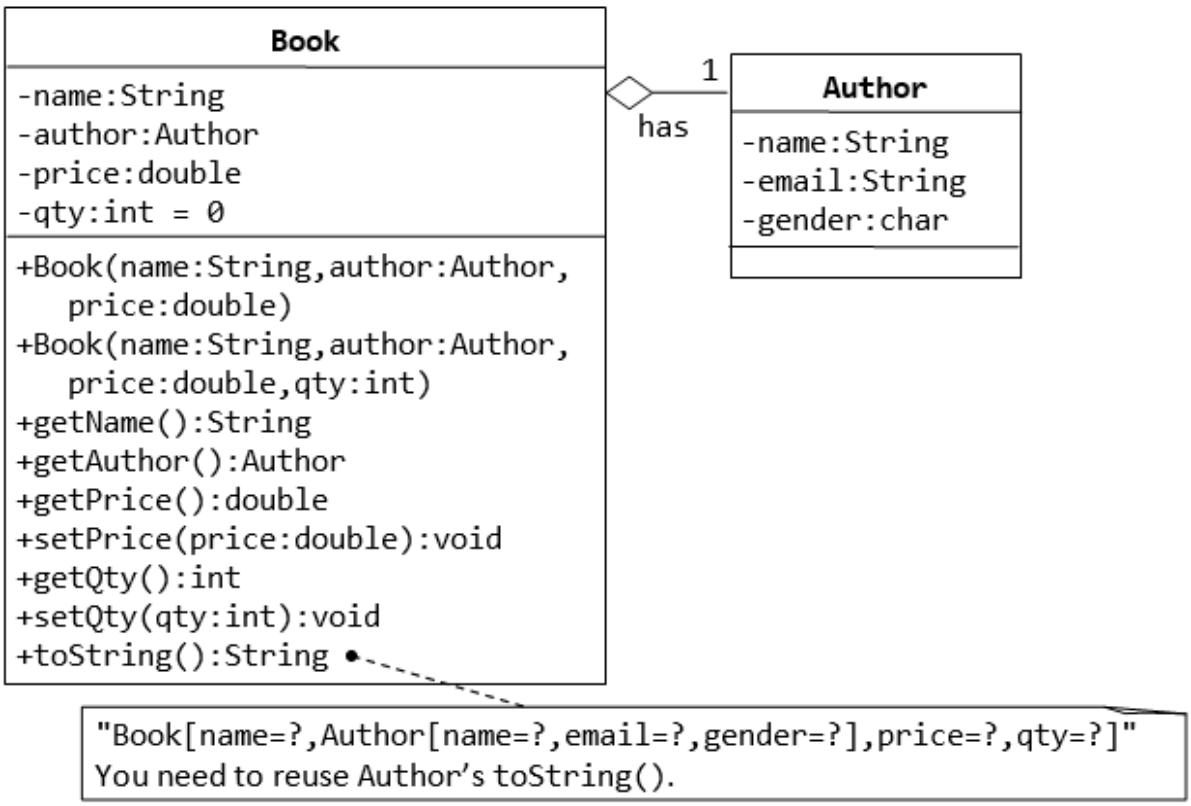
14. Develop a java application for an automobile shop which sells used cars. The application maintains details about the car such as name, model, year of purchase, color, and price.
    Maintain a database for each of the car's objects. For example: find the initial 5 car objects details
    1. "Brezza", "VX", "02/10/2017", "White", INR 6,50,500.00
    2. "Venue", "SX", "20/9/2018", "White", INR 7,50,000.00
    3. "Ciaz", "VX", "15/12/2015", "White", INR 6,00,000.00
    4. "Ecosport", "LX", "21/6/2018", "Blue", INR 5,00,500.00
    5. "Nexon", "SX", "15/10/2019", "Green", INR 8,50,500.00

    Application automatically removes the car details from the database once the booking is done.

    Define a class "Car", include all data members and methods. Include getter, setter functions for all the data members. In order to provide the car details, override the function toString(). Write a testing class (main class) to validate each function.

Lab4

15.



A class called Book is designed (as shown in the class diagram) to model a book written by *one* author. It contains:

- Four private instance variables: name (String), author (of the class Author you have just created, assume that a book has one and only one author), price (double), and qty (int);
- Two constructors:

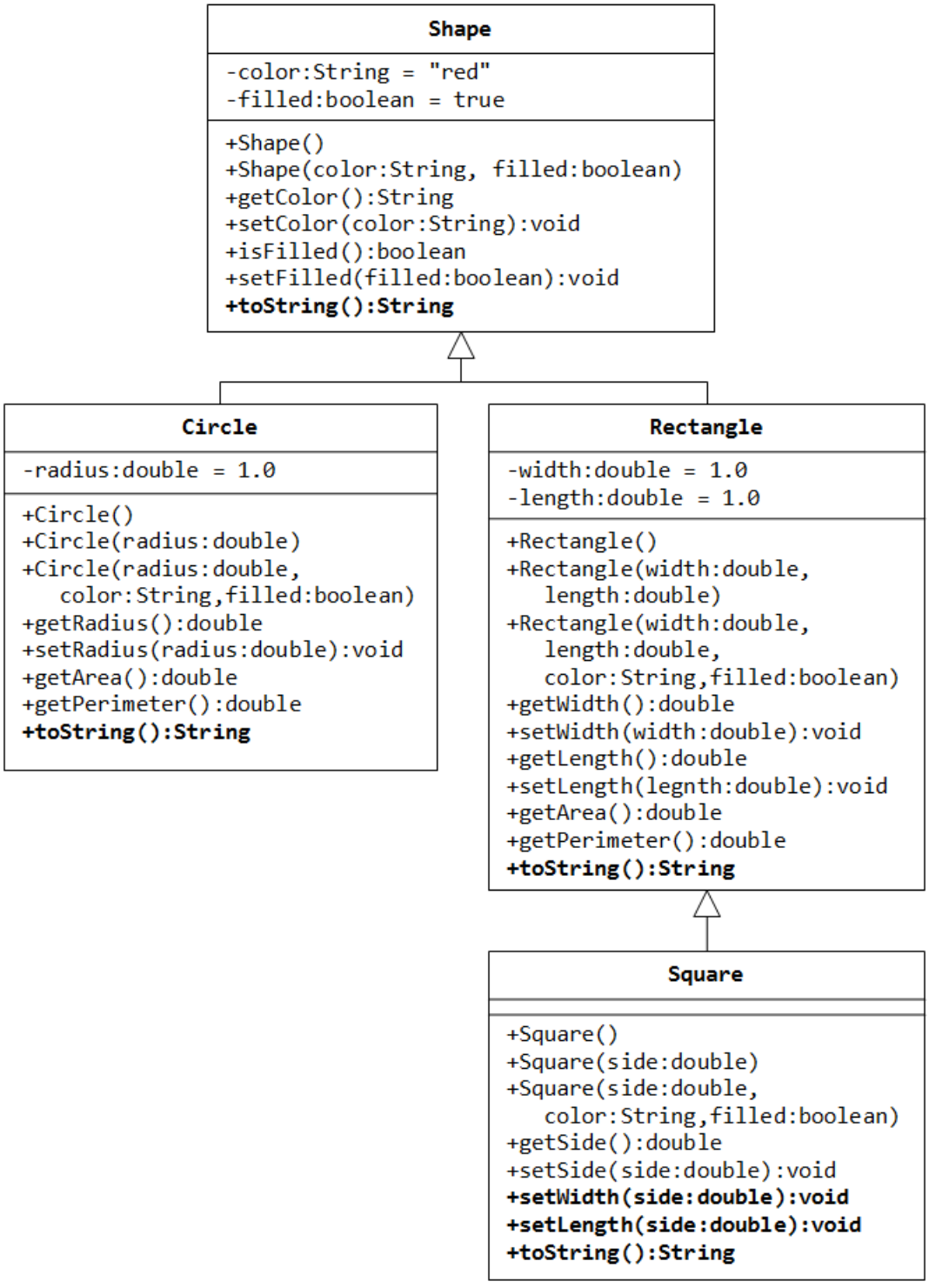
```
public Book (String name, Author author, double price) { ..... }
public Book (String name, Author author, double price, int qty) { ..... }
```

- public methods getName(), getAuthor(), getPrice(), setPrice(), getQty(), setQty().
- A toString() that returns "Book[name=?, Author[name=?,email=?, gender=?], price=?, qty=?". You should reuse Author's toString().

Write the Book class (which uses the Author class). Also write a test driver called TestBook to test all the public methods in the class Book. Take Note that you have to construct an instance of Author before you can construct an instance of Book.

16. Develop Stack with following operations
    i) createNew( ) ii) push( ) iii) pop( ) iv) display( ) v) empty( )

17.



Write a superclass called Shape (as shown in the class diagram), which contains:

- Two instance variables color (String) and filled (boolean).
- Two constructors: a no-arg (no-argument) constructor that initializes the color to "green" and filled to true, and a constructor that initializes the color and filled to the given values.

- Getter and setter for all the instance variables. By convention, the getter for a boolean variable xxx is called isXXX() (instead of getXxx() for all the other types).
- A toString() method that returns "A Shape with color of xxx and filled/Not filled".

Write a test program to test all the methods defined in Shape.
Write two subclasses of Shape called Circle and Rectangle, as shown in the class diagram.

The Circle class contains:

- An instance variable radius (double).
- Three constructors as shown. The no-arg constructor initializes the radius to 1.0.
- Getter and setter for the instance variable radius.
- Methods getArea() and getPerimeter().
- Override the toString() method inherited, to return "A Circle with radius=xxx, which is a subclass of yyy", where yyy is the output of the toString() method from the superclass.

The Rectangle class contains:

- Two instance variables width (double) and length (double).
- Three constructors as shown. The no-arg constructor initializes the width and length to 1.0.
- Getter and setter for all the instance variables.
- Methods getArea() and getPerimeter().
- Override the toString() method inherited, to return "A Rectangle with width=xxx and length=zzz, which is a subclass of yyy", where yyy is the output of the toString() method from the superclass.

Write a class called Square, as a subclass of Rectangle. Convince yourself that Square can be modeled as a subclass of Rectangle. Square has no instance variable, but inherits the instance variables width and length from its superclass Rectangle.

- Provide the appropriate constructors (as shown in the class diagram). Hint:

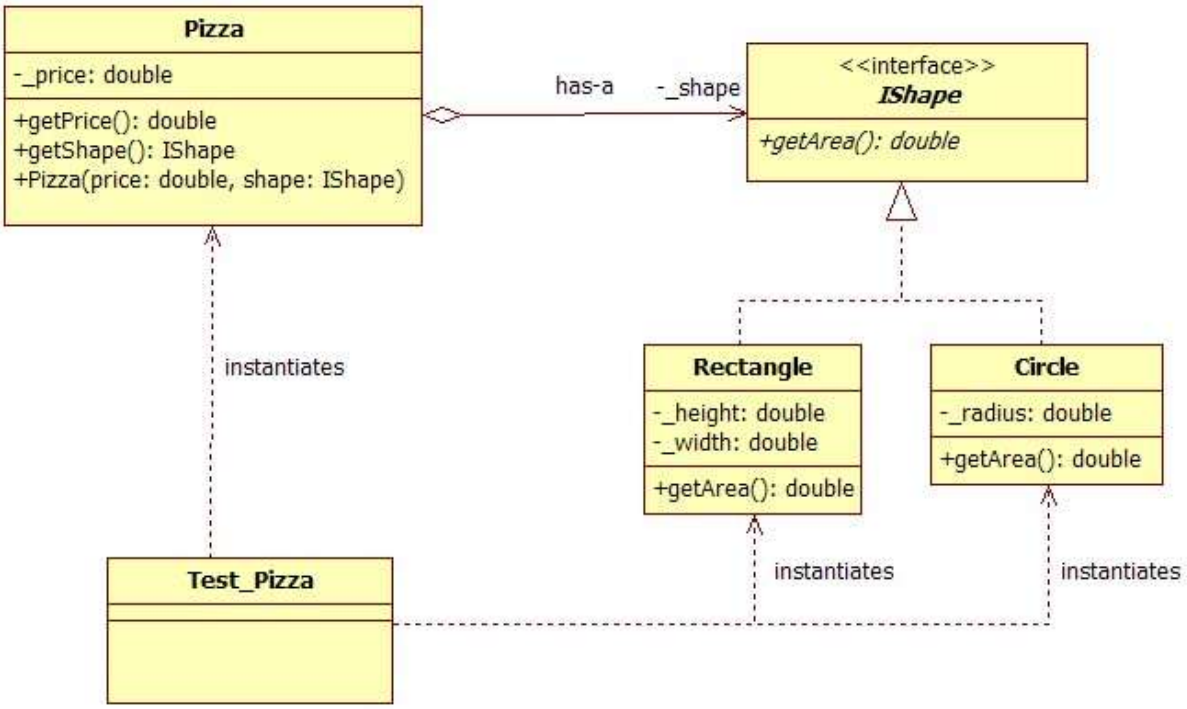
```
public Square(double side) {
  super(side, side);  // Call superclass Rectangle(double, double)
}
```

- Override the toString() method to return "A Square with side=xxx, which is a subclass of yyy", where yyy is the output of the toString() method from the superclass.
- Do you need to override the getArea() and getPerimeter()? Try them out.
- Override the setLength() and setWidth() to change both the width and length, so as to maintain the square geometry.

Lab5

18. Refer to the diagram below and implement the java application. Identify the various classes and interface, accordingly, implement each of them.

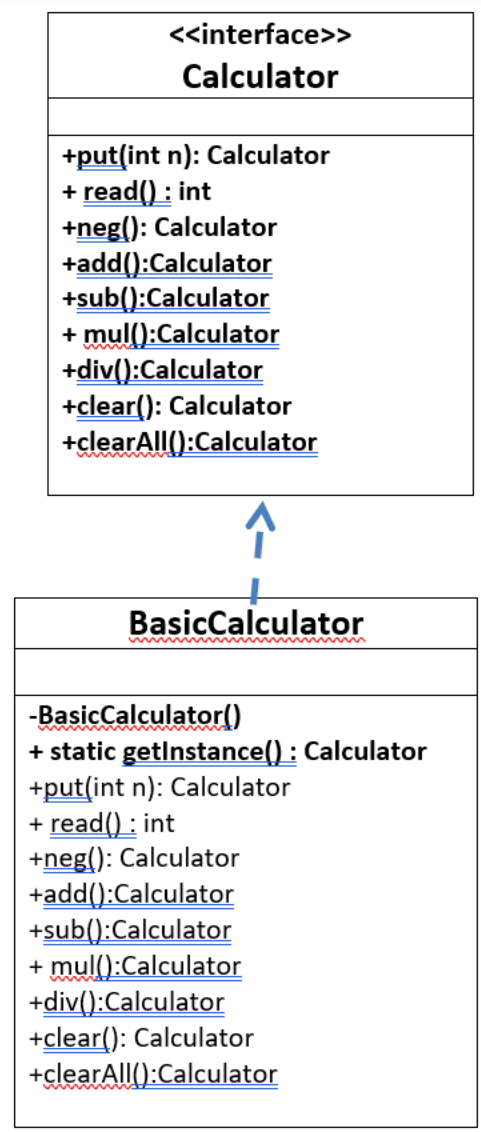
    Write the few test cases in the test class (Test_Pizza) to validate Pizza's shape, dimensions and price.



19. Write an interface called Calculator (as shown in the diagram), which contains all basic operations of a calculator.

    Write a class called BasicCalculator, which uses interface Calculator. Implement all the methods in the class which are available in an interface. The class BasciCalculator has a static function called getInstance, which will return an object of type Calculator.

    Write the test cases to validate all the arithmetic operations.



Lab6

20. Design and implement console-based Java application for the following requirements.

1. Write a login program. Read stored user credentials from a text file (login.txt) and authenticate the user. For a valid user, load main program or else display invalid user. Give three chances to provide credentials.
2. For a valid user, load main program which contains menu items such as insert, delete, display.
3. On selecting **Insert** menu item, write a new student details into a text file ("**student_details.txt**").
4. On selecting **Delete** menu item, display only students' name. On selecting a specific student's name, remove specific record from **student_details text** file.
5. On selecting **Display** menu item, display all the records from **student_details text** file.